

Paxos vs Raft: 我们在分布式共识上达成共识了吗?

Heidi Howard 剑
桥大学 英国剑桥 fir
st.last@cl.cam.ac.uk

理查德·莫蒂尔 (R
ichard Mortier) 剑桥
大学 英国剑桥 first.
last@cl.cam.ac.uk

抽象的

分布式共识是构建容错、强一致性分布式系统的基本术语。尽管已经提出了许多分布式共识算法，但只有两种主导生产系统：Paxos，传统的、以微妙著称的算法；Raft，一种更新的算法，定位为 Paxos 的更易于理解的替代方案。

在本文中，我们考虑以下问题：Paxos 或 Raft 哪种算法是分布式共识的更好解决方案？我们通过使用 Raft 的术语和实用抽象描述简化的 Paxos 算法来分析两者，以确定它们到底有何不同。

我们发现 Paxos 和 Raft 都采用非常相似的分布式共识方法，唯一的区别在于领导者选举的方法。最值得注意的是，Raft 只允许具有最新日志的服务器成为领导者，而 Paxos 允许任何服务器成为领导者，只要它更新其日志以确保其是最新的。Raft 的方法非常简单，其效率令人惊讶，因为与 Paxos 不同，它不需要在领导者选举期间交换日志条目。我们推测 Raft 的可理解性很大程度上来自于论文的清晰呈现，而不是所呈现的底层算法的基础。

CCS 概念： • 计算机系统组织 → 可靠性；
• 软件及其工程 → 云计算；
• 计算理论 → 分布式算法。

关键词：状态机复制、分布式共识、Paxos、Raft

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. PaPoC '20, April 27, 2020, Heraklion, Greece

© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-7524-5/20/04...\$15.00

<https://doi.org/10.1145/3380787.3393681>

ACM参考格式：

海蒂·霍华德和理查德·莫蒂尔。2020. Paxos vs Raft: 分布式共识达成共识了吗? 。第七届分布式数据一致性原则与实践研讨会 (PaPoC '20), 2020 年 4 月 27 日, 希腊伊拉克利翁。ACM, 美国纽约州纽约市, 9 页。 <https://doi.org/10.1145/3380787.3393681>

1 简介

状态机复制[32]被广泛用于将一组不可靠的主机组合成单个可靠的服务，该服务可以提供包括线性化在内的强一致性保证[13]。因此，程序员可以将使用复制状态机实现的服务视为单个系统，从而轻松推断预期行为。状态机复制要求每个状态机以相同的顺序接收相同的操作，这可以通过分布式共识来实现。

Paxos 算法 [16] 是分布式共识的代名词。尽管 Paxos 取得了成功，但众所周知，它很难理解，因此很难推理、正确实施和安全优化。这在用更简单的术语解释算法的大量尝试中很明显[4,17,22,23,25,29,35]，并且是Raft[28]背后的动机。

Raft 的作者声称 Raft 与 Paxos 一样高效，同时更易于理解，从而为构建实用系统提供了更好的基础。Raft 寻求通过三种不同的方式来实现这一目标：

首先，Raft 论文引入了一种新的抽象，用于描述状态机复制背景下基于领导者的共识。事实证明，这种务实的演讲非常受工程师欢迎。

简单性 其次，Raft 论文优先考虑简单性而不是性能。例如，Raft 按顺序决定日志条目，而 Paxos 通常允许无序决策，但需要额外的协议来填充可能出现的日志间隙。

底层算法 最后，Raft 算法采用了一种新颖的领导者选举方法，它改变了领导者选举的方式。

领导者是选举出来的，因此如何保证安全。Raft 迅速流行起来 [30]，如今的生产系统分为使用 Paxos [3,5,31,33,36,38] 和使用 Raft [2,8-10,15,24,34] 的系统。

为了回答 Paxos 和 Raft 哪个是分布式共识的更好解决方案的问题，我们必须首先回答这两种算法到底有什么不同。

达成共识的方法？这不仅有助于评估这些算法，还可以让 Raft 从数十年来优化 Paxos 性能的研究中受益 [6, 12, 14, 18–20, 26, 27]，反之亦然 [1, 37]。

然而，回答这个问题并不是一件简单的事情。Paxos 通常不被视为单一算法，而是被视为解决分布式共识的一系列算法。Paxos 的通用性（或不规范，取决于您的观点）意味着算法的描述因论文而异，有时甚至相当大。

为了克服这个问题，我们在此提出了 Paxos 的简化版本，该版本是通过调查 Paxos 的各种已发布描述而得出的。这个算法，我们简称为 Paxos，更符合当今 Paxos 的使用方式，而不是它最初的描述方式 [16]。它在其他地方被称为多法令 Paxos，或简称 MultiPaxos，以区别于单法令 Paxos，后者决定单个值而不是全序值序列。我们还使用 Raft 论文中的风格和抽象来描述我们的简化算法，从而可以对两种不同的算法进行公平的比较。

我们的结论是，算法之间的可理解性没有显著差异，而且考虑到 Raft 的简单性，其领导者选举的效率令人惊讶。

2 背景

本文研究了状态机复制背景下的分布式共识。状态机复制要求应用程序的确定性状态机在 n 个服务器之间复制，每个服务器以相同的顺序应用相同的操作集。这是通过使用复制日志来实现的，由分布式共识算法（通常是 Paxos 或 Raft）管理。

我们假设系统是非拜占庭的[21]，但我们不假设系统是同步的。消息可能会被任意延迟，参与的服务器可能会以任何速度运行，但我们假设消息交换是可靠且有序的（例如，通过使用 TCP/IP）。我们不依赖时钟同步来保证安全，尽管我们必须依赖时钟同步来保证活性[11]。我们假设 n 个服务器中的每一个都有一个唯一的 id s ，其中 $s \in \{0..(n-1)\}$ 。我们假设操作是唯一的，通过向每个操作添加一对序列号和服务器 ID 即可轻松实现。

3 Paxos和Raft的实现方法

许多共识算法，包括 Paxos 和 Raft，都使用基于领导者的方法来解决分布式共识。在高层次上，这些算法的运行方式如下：

n 个服务器之一被指定为领导者。状态机的所有操作都发送给领导者。领导者将操作附加到其日志中，并要求其他服务器执行相同操作。一旦领导者收到大多数服务器的确认，这已经发生，

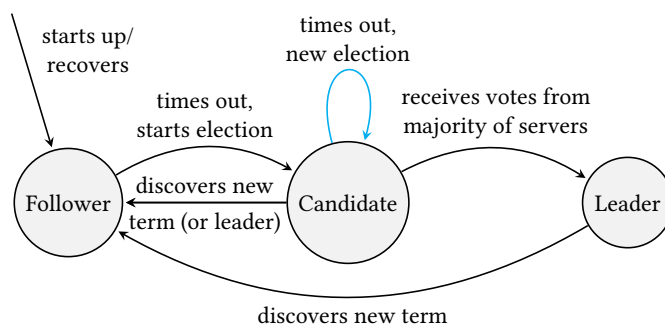


图 1. Paxos 和 Raft 的服务器状态之间的状态转换。蓝色的过渡是 Raft 特有的。

它将操作应用于其状态机。重复这个过程直到领导者失败。当领导者失败时，另一台服务器将接替领导者。这个选举新领导者的过程至少涉及大多数服务器，确保新领导者不会覆盖任何先前应用的操作。

我们现在更详细地研究 Paxos 和 Raft。读者可能会发现参考附录 A 和 B 中提供的 Paxos 和 Raft 的总结会有所帮助。这里我们重点关注 Paxos 和 Raft 的核心元素，由于篇幅限制，不比较垃圾收集、日志压缩、读操作或重新配置算法。

3.1 基础知识

如图 1 所示，服务器在任何时候都可能处于以下三种状态之一：

Follower 一种被动状态，仅负责回复 RPC。

候选者 一种活动状态，尝试使用 RequestVotes RPC 成为领导者。

Leader 一种活动状态，负责使用 AppendEntries RPC 将操作添加到复制日志。

最初，服务器处于跟随者状态。每个服务器继续作为追随者，直到它认为领导者失败。然后，追随者成为候选人，并尝试使用 RequestVote RPC 当选领导者。如果成功，候选人将成为领导者。新领导者必须定期发送 AppendEntries RPC 作为 keepalive，以防止追随者超时并成为候选人。

每个服务器存储一个自然数，即术语，它随着时间的推移单调增加。最初，每个服务器的当前项为零。发送服务器（以下称为发送方）的当前术语包含在每个 RPC 中。当服务器收到 RPC 时，它（以下称为服务器）首先检查包含的术语。如果发送者的任期大于服务器的任期，则服务器将在响应 RPC 之前更新其任期，并且如果服务器是候选者或领导者，则步骤

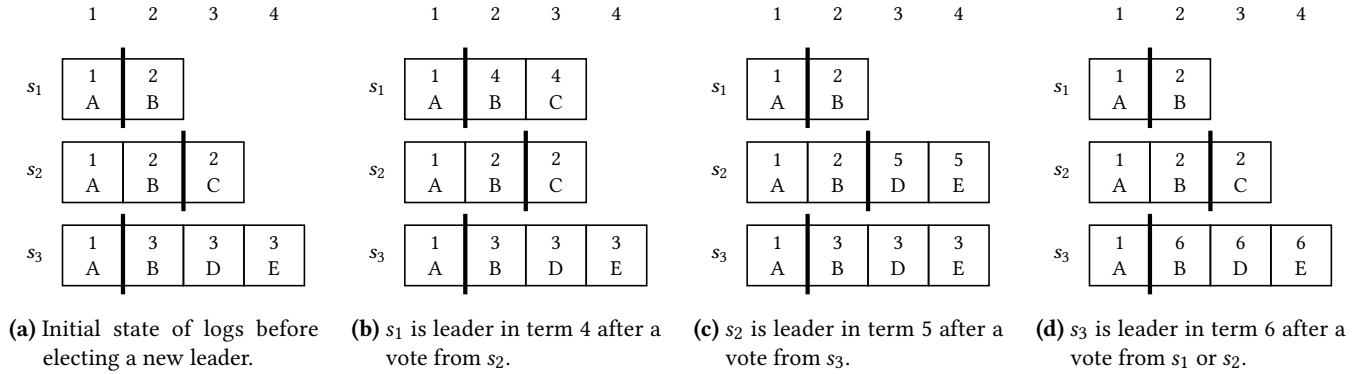


图 2. 运行 Paxos 的三台服务器的日志。图(a)显示了触发leader选举时的日志。图 (b—d) 显示了领导者当选后但发送第一个 AppendEntries RPC 之前的日志。黑线显示提交索引，红色文本突出显示日志更改。

下来成为追随者。如果发送者的期限与服务器的期限相同，则服务器将照常响应 RPC。如果发送者的术语小于服务器的术语，则服务器将对发送者做出否定响应，并将其术语包含在响应中。当发送者收到这样的响应时，它将降级为追随者并更新其术语。

3.2 正常运行

当领导者收到操作时，它会将其与当前术语一起附加到其日志的末尾。这对操作和术语称为日志条目。然后，领导者将带有新日志条目的 AppendEntries RPC 发送到所有其他服务器。每个服务器维护一个提交索引来记录哪些日志条目可以安全地应用于其状态机，并响应领导者确认成功收到新日志条目。一旦领导者收到来自大多数服务器的肯定响应，领导者就会更新其提交索引并将操作应用于其状态机。然后，领导者将更新后的提交索引包含在后续的 AppendEntries RPC 中。

仅当追随者在该条目（或多个条目）之前的日志与领导者的日志相同时，才会追加该日志条目（或一组日志条目）。这可确保日志条目按顺序添加，防止日志中出现间隙，并确保关注者将正确的日志条目应用到其状态机。

3.3 处理领导失败

这个过程一直持续到领导者失败，需要建立新的领导者。Paxos 和 Raft 对这个过程采用不同的方法，因此我们依次描述它们。

帕克索斯。如果追随者未能从领导者那里收到最近的 AppendEntries RPC，则会超时。然后它成为候选人并将其术语更新为下一个术语，使得 $t \bmod n = s$ 其中 t 是下一个术语， n 是服务器数量， s 是候选者的服务器 ID。候选人将 RequestVote RPC 发送到其他服务器。该 RPC 包括候选人的新术语和提交索引。什么时候

服务器收到 RequestVote RPC，如果候选人的任期大于自己的任期，它将做出积极响应。此响应还包括服务器在其日志中位于候选提交索引之后的任何日志条目。

一旦候选人收到来自大多数服务器的肯定的 RequestVote 响应，候选人必须确保其日志包含所有已提交的条目，然后才能成为领导者。其操作如下。对于提交索引之后的每个索引，领导者会审查它收到的日志条目以及自己的日志。如果候选者看到了索引的日志条目，那么它将使用该条目和新术语更新自己的日志。如果领导者看到同一索引的多个日志条目，那么它将使用最大术语和新术语中的条目更新自己的日志。图 2 中给出了一个示例。候选者现在可以成为领导者并开始将其日志复制到其他服务器。

筏。至少有一个追随者在没有收到领导者最近的 AppendEntries RPC 后会超时。它将成为候选人并增加其任期。候选人将 RequestVote RPC 发送到其他服务器。每个都包括候选者的术语以及候选者的最后一个日志术语和索引。当服务器收到 RequestVote 请求时，如果候选人的任期大于或等于自己的任期，服务器在该任期内尚未投票给候选人，并且候选人的日志至少与自己的日志一样最新，它将积极响应。可以通过确保候选者的最后一个对数项大于服务器的，或者如果它们相同，则候选者的最后一个索引大于服务器的来检查最后一个标准。

一旦候选人收到来自大多数服务器的肯定的 RequestVote 响应，候选人就可以成为领导者并开始复制其日志。然而，为了安全起见，Raft 要求领导者在提交新术语中的至少一个日志条目之前不要更新其提交索引。

由于给定任期内可能有多名候选人，因此选票可能会分散，导致没有候选人获得多数票。在这种情况下，候选人超时并从下一个任期开始新的选举。

3.4 安全

两种算法都保证以下属性：

定理 3.1（状态机安全）。如果服务器已将给定索引处的日志条目应用到其状态机，则其他服务器将不会为同一索引应用不同的日志条目。

每个任期最多有一个领导者，并且领导者不会覆盖自己的日志，因此我们可以通过证明以下内容来证明这一点：

定理 3.2（领导者完整性）。如果项 t 中的领导者在索引 i 处提交了操作 op ，则项 $> t$ 的所有领导者也将在索引 i 处拥有操作 op 。

Paxos 的证明草图。假设操作 op 在索引 i 处提交，期限为 t_0 。我们将对大于 t 的项使用归纳证明。

基本情况：如果存在项 $t + 1$ 的领导者，则它将在索引 i 处执行操作 op 。

由于任意两个多数仲裁相交并且消息按 term 排序，因此至少一台在索引 i 处具有操作 op 且具有 term t 的服务器必须积极响应来自 $t + 1$ 领导者的 RequestVote RPC。该服务器无法删除或覆盖此操作，因为它无法积极响应自 term t 领导者以来任何其他领导者的 AppendEntries RPC。领导者将选择操作 op ，因为它不会收到任何带有术语 $> t$ 的日志条目。

归纳情况：假设项 $t + 1$ 到 $t + k$ 的任何前导项在索引 i 处都有操作 op 。如果存在项 $t + k + 1$ 的领导者，那么它也将索引 i 处具有操作 op 。

由于任意两个多数群体相交并且消息按术语排序，因此至少一台在索引 i 处具有操作 op 且术语 t 到 $t + k$ 的服务器必须积极回复来自术语 $t + k + 1$ 的领导者的 RequestVote RPC。这是因为服务器无法删除或覆盖此操作，因为它没有积极响应来自任何领导者的 AppendEntries RPC，除了术语 t 到 $t + k$ 的领导者之外。根据我们的归纳假设，所有这些领导者也将在索引 i 处进行操作 op ，因此不会覆盖它。仅当领导者在索引 i 处接收到具有该不同操作且具有更大术语的日志条目时，才可以选择另一个操作。根据我们的归纳假设，项 t 到 $t + k$ 的所有领导者也将在索引 i 处具有操作 op ，因此不会在索引 i 处写入另一个操作。□

Raft 的证明使用相同的归纳法，但由于 Raft 的领导者选举方法不同，细节有所不同。

4 讨论

Raft 和 Paxos 采用不同的领导者选举方法，如表 1 所示。我们比较两个维度：可理解性和效率，以确定哪种方法最好。

易懂。Raft 保证如果两个日志包含相同的操作，那么两个日志将具有相同的索引和术语。换句话说，每个操作都被分配了一个唯一的索引和术语对。然而，Paxos 中的情况并非如此，未来的领导者可能会为某个操作分配更高的任期，如图 2b 中的操作 B 和 C 所示。在 Paxos 中，提交索引之前的日志条目可能会被覆盖。这是安全的，因为日志条目只会被具有相同操作的条目覆盖，但它不像 Raft 的方法那么直观。

另一方面是，如果大多数服务器上都存在日志条目，Paxos 可以安全地提交该日志条目。但 Raft 的情况并非如此，它要求领导者仅提交前一个任期的日志条目（如果该日志条目存在于大多数服务器上，并且领导者已提交当前任期的后续日志条目）。

在 Paxos 中，领导者复制的日志条目要么来自当前任期，要么已经提交。我们可以在图 2 中看到这一点，其中领导者提交索引之后的所有日志条目都具有当前术语。Raft 中的情况并非如此，领导者可能会复制先前术语中未提交的条目。

总体而言，我们认为 Raft 的方法比 Paxos 的方法稍微更容易理解，但并不明显。

效率。在 Paxos 中，如果多个服务器同时成为候选人，则任期较高的候选人将赢得选举。在 Raft 中，如果多个服务器同时成为候选人，由于任期相同，它们可能会分散选票，因此都不会赢得选举。Raft 通过让追随者在选举超时后等待一段额外的时间（从均匀随机分布中抽取）来缓解这种情况。因此，我们预计 Raft 会更慢，并且在选举领导者所需的时间上有更大的方差。

然而，Raft 的领导者选举阶段比 Paxos 更轻量。Raft 只允许拥有最新日志的候选者成为领导者，因此在领导者选举期间不需要发送日志条目。Paxos 的情况并非如此，其中每个积极的 RequestVote 响应都在候选人的提交索引之后包含关注者的日志条目。有多种选项可以减少发送的日志条目的数量，但最终，如果领导者的日志尚未更新，则始终需要发送一些日志条目。

Paxos 不仅仅通过 RequestVote 响应发送比 Raft 更多的日志条目。在这两种算法中，一旦候选人成为领导者，它就会将其日志复制到所有其他服务器。在 Paxos 中，领导者可能已经给日志条目赋予了新术语，因此领导者可以将日志条目的另一个副本发送到已经拥有副本的服务器。

	Paxos	Raft
How does it ensure that each term has at most one leader?	A server s can only be a candidate in a term t if $t \bmod n = s$. There will only be one candidate per term so only one leader per term.	A follower can become a candidate in any term. Each follower will only vote for one candidate per term, so only one candidate can get a majority of votes and become the leader.
How does it ensure that a new leader's log contains all committed log entries?	Each RequestVote reply includes the follower's log entries. Once a candidate has received RequestVote responses from a majority of followers, it adds the entries with the highest term to its log.	A vote is granted only if the candidate's log is at least as up-to-date as the followers'. This ensures that a candidate only becomes a leader if its log is at least as up-to-date as a majority of followers.
How does it ensure that leaders safely commit log entries from previous terms?	Log entries from previous terms are added to the leader's log with the leader's term. The leader then replicates the log entries as if they were from the leader's term.	The leader replicates the log entries to the other servers without changing the term. The leader cannot consider these entries committed until it has replicated a subsequent log entry from its own term.

表 1. Paxos 和 Raft 之间的差异总结

Raft 的情况并非如此, 每个日志条目在日志中的整个时间内都保留相同的术语。同样, Paxos 中有多种缓解此问题的选项, 但它们超出了我们简化的 Paxos 算法的范围。

总体而言, 与 Paxos 相比, Raft 的领导者选举方法对于如此简单的方法来说出奇地高效。

5 与经典 Paxos 的关系

熟悉 Paxos 的读者可能会觉得我们对 Paxos 的描述与之前发布的有所不同, 因此我们现在概述我们的 Paxos 算法与文献中其他地方发现的算法之间的关系。

角色。Paxos 的一些描述将 Paxos 的职责分为三个角色: 提议者、接受者和学习者 [17] 或领导者、接受者和复制者 [35]。我们的演示仅使用一种角色, 即服务器, 它包含所有角色。这个使用单个角色的演示也使用了名称副本[7],

术语。术语也称为视图、选票号[35]、提案号[17]、轮数、序列号[7]或纪元。我们的领导者也被称为大师[7]、主要者、协调者或杰出的提议者[17]。通常, 服务器作为候选者的时期称为阶段 1, 服务器作为领导者的时期称为阶段 2。RequestVote RPC 通常称为 Phase1a 和 Phase1b 消息 [35]、准备请求和响应 [17] 或准备和承诺消息。AppendEntries RPC 通常称为 Phase2a 和 Phase2b 消息 [35]、接受请求和响应 [17] 或提议和接受消息。

条款。Paxos 只要求术语完全排序, 并且为每个服务器分配一组不相交的术语 (为了安全), 并且每个服务器可以使用比任何其他术语都大的术语 (为了活性)。虽然一些描述

Paxos 像我们一样使用循环自然数 [7], 其他使用按字典顺序排列的对, 由整数和服务器 ID 组成, 其中每个服务器仅使用包含自己 ID 的术语 [35]。

订购。我们的日志条目按顺序复制和决定。这不是必需的, 但它确实避免了填充日志间隙的复杂性[17]。类似地, Paxos 的一些描述限制了并发决策的数量, 这通常是重新配置所必需的 [17, 35]。

6 总结

Raft 算法的提出是为了解决广泛研究的 Paxos 算法长期存在的可理解性问题。

在本文中, 我们证明了 Raft 的可理解性很大程度上来自于其务实的抽象和出色的表达。通过使用与 Raft 相同的方法描述简化的 Paxos 算法, 我们发现这两种算法仅在领导者选举方法上有所不同。具体来说:

- (i) Paxos 在服务器之间划分任期, 而 Raft 允许追随者成为任何任期的候选人, 但追随者每个任期只能投票给一名候选人。
- (ii) Paxos 追随者将投票给任何候选人, 而 Raft 追随者只会投票给候选人日志至少是最新的候选人。
- (iii) 如果领导者在当前任期内有未提交的日志条目, Paxos 将在当前任期中复制它们, 而 Raft 将在原始任期中复制它们。

The Raft paper claims that Raft is significantly more understandable than Paxos, and as efficient.相反, 我们发现这两种算法在可理解性上没有显著差异, 但与 Paxos 相比, Raft 的领导人选举出人意料地轻量级。这两种算法我们

已经提出的设计很幼稚，当然可以进行优化以提高性能，尽管通常以增加复杂性为代价。

参考

[1] Vaibhav Arora, Tanuj Mittal, Divyakant Agrawal, Amr El Abbadi, Xun Xu, Zhiyananzhiyanan, Zhujiangfeng Zhujiangfeng. 2017. 领导者或多数：当你可以两者兼得时，为什么只拥有一个？提高 Raft 类共识协议的读取可扩展性。第九届 USENIX 云计算热门主题会议（加利福尼亚州圣克拉拉）会议记录 (HotCloud' 17)。USENIX 协会，美国，14。[2] atomix [n.d.]. 阿托米克斯。https://atomix.io。[3] Jason Baker、Chris Bond、James C. Corbett、JJ Furman、Andrey Khorlin、James Larson、Jean-Michel Leon、Yawei Li、Alexander Lloyd 和 Vadim Yushpakh. 2011. Megastore：为交互式服务提供可扩展、高可用的存储。创新数据系统研究会议 (CIDR) 会议记录。223–234。http://www.cidrdb.org/cidr2011/Papers/CIDR11_Paper32.pdf [4] Romain Boichat, Partha Dutta, Svend Frølund 和 Rachid Guerraoui. 2003. 解构 Paxos. SIGACT 新闻 34, 1 (2003 年 3 月), 47–67。https://doi.org/10.1145/637437.637447 [5] 迈克·伯罗斯。2006. 松耦合分布式系统的 Chubby Lock 服务。第七届操作系统设计与实现研讨会（华盛顿州西雅图）(OSDI' 06) 论文集。USENIX 协会，美国，335–350。[6] 拉萨罗·乔纳斯·卡马戈斯、罗德里戈·马耳他·施密特和费尔南多·佩多内。2007. 多协调 Paxos。摘自第二十六届 ACM 分布式计算原理年度研讨会（美国俄勒冈州波特兰）(PODC' 07)。计算机协会，美国纽约州纽约市，316–317。https://doi.org/10.1145/1281100.1281150 [7] Tushar D. Chandra、Robert Griesemer 和 Joshua Redstone. 2007 年。Paxos 上线：工程视角。第二十六届 ACM 分布式计算原理年度研讨会（美国俄勒冈州波特兰）(PODC' 07) 论文集。计算机协会，美国纽约州纽约市，398–407。https://doi.org/10.1145/1281100.1281103 [8] cockroachdb [n.d.]. 蟑螂数据库。https://www.cockroachlabs.com。[9] 领事[n.d.]. Hash iCorp 的领事。https://www.consul.io。[10] etcd[n.d.]. 等等。https://coreos.com/etcd/。[11] 迈克尔·J·费舍尔、南希·A·林奇和迈克尔·S·帕特森。1985 年。通过一个错误的流程不可能达成分布式共识。J. ACM 32, 2 (1985 年 4 月), 374–382。https://doi.org/10.1145/3149.214121 [12] 伊莱·加夫尼和莱斯利·兰波特。2000. 磁盘 Paxos。第 14 届国际分布式计算会议 (DISC' 00) 的会议记录。施普林格出版社，柏林，海德堡，330–344。[13] 莫里斯·P·赫利希和珍妮特·M·温。1990. 线性化：并发对象的正确性条件。ACM 翻译。程序。郎。系统。12, 3 (1990 年 7 月), 463–492。https://doi.org/10.1145/78969.78972 [14] Tim Kraska, Gene Pang, Michael J. Franklin, Samuel Madden 和 Alan Fekete. 2013. MDCC：多数据中心一致性。第八届 ACM 欧洲计算机系统会议（捷克共和国布拉格）会议记录 (EuroSys '13)。计算机协会，美国纽约州纽约市，113–126。https://doi.org/10.1145/2465351.2465363 [15] Kubernetes [n.d.]. Kubernetes：生产级容器编排。https://kubernetes.io。[16] 莱斯利·兰波特。1998 年。非全日制议会。ACM 翻译。计算。系统。16, 2 (1998 年 5 月), 133–169。https://doi.org/10.1145/279227.279229 [17] 莱斯利·兰波特。2001. Paxos 变得简单。ACM SIGACT 新闻（分布式计算专栏）32, 4 (第 121 期, 2001 年 12 月) (2001 年 12 月), 51–58。https://www.microsoft.com/en-us/research/publication/paxos-made-simple/

[18] 莱斯利·兰波特。2005. 广义共识和 Paxos。技术报告 MSR-TR-2005-33。微软。60 页。https://www.microsoft.com/en-us/research/publication/generalized-consensus-and-paxos/ [19] 莱斯利·兰波特。2006。快速 Paxos。分布式计算 19 (2006 年 10 月), 79–103。https://www.microsoft.com/en-us/research/publication/fast-paxos/ [20] 莱斯利·兰波特和迈克·马萨。2004. 廉价的 Paxos。2004 年国际可靠系统和网络会议 (DSN '04) 会议记录。IEEE 计算机协会，美国，307。[21] Leslie Lamport、Robert Shostak 和 Marshall Pease。1982. 拜占庭将军问题。ACM 翻译。程序。郎。系统。4, 3 (1982 年 7 月), 382–401。https://doi.org/10.1145/357172.357176 [22] 巴特勒·兰普森。2001. Paxos 的 ABCD。第二十届 ACM 分布式计算原理年度研讨会（美国罗德岛州纽波特）(PODC' 01) 论文集。计算机协会，美国纽约州纽约市，13。https://doi.org/10.1145/383962.383969 [23] 巴特勒·W·兰普森。1996。如何使用共识构建高度可用的系统。第十届国际分布式算法研讨会 (WDAG '96) 论文集。Springer-Verlag, 柏林，海德堡，1–17。[24] m3 [n.d.]. M3: Uber 的 Prometheus 开源大规模指标平台。https://eng.uber.com/m3/。[25] 海因·梅林和利安德·杰尔。2013. 教程摘要：从头开始解释 Paxos。第 17 届分布式系统原理国际会议论文集 - 第 8304 卷（法国尼斯）(OPODIS 2013)。施普林格出版社，柏林，海德堡，1–10。https://doi.org/10.1007/978-3-319-03850-6_1 [26] Iulian Moraru、David G. Andersen 和 Michael Kaminsky. 2013 年。平等主义议会会有更多共识。第二十四届 ACM 操作系统原理研讨会论文集（宾夕法尼亚州法明顿）(SOSP' 13)。计算机协会，美国纽约州纽约市，358–372。https://doi.org/10.1145/2517349.2517350 [27] Iulian Moraru、David G. Andersen 和 Michael Kaminsky. 2014. Paxos Quorum Leases：快速读取而不牺牲写入。ACM 云计算研讨会论文集（美国华盛顿州西雅图）(SOCC' 14)。计算机协会，美国纽约州纽约市，1–13。https://doi.org/10.1145/2670979.2671001 [28] 迭戈·翁加罗和约翰·奥斯特豪特。2014. 寻找一种可理解的共识算法。2014 年 USENIX 年度技术会议（宾夕法尼亚州费城）会议记录 (USENIX ATC' 14)。USENIX 协会，美国，305–320。[29] 罗伯托·德普里斯科、巴特勒·W·兰普森和南希·A·林奇。1997 年。重新审视 Paxos 算法。第 11 届国际分布式算法研讨会 (WDAG '97) 论文集。施普林格出版社，柏林，海德堡，111–125。[30] 筏[n.d.]. Raft 共识算法。https://raft.github.io。[31] Raghu Ramakrishnan、Baskar Sridharan、John R. Douceur、Pavan Kasturi、Balaji Krishnamachari-Sampath、Karthick Krishnamoorthy、Peng Li、Mitica Manu、Spiro Michaylov、Rogério Ramos 等。2017 年。Azure Data Lake Store：用于大数据分析的超大规模分布式文件服务。2017 年 ACM 国际数据管理会议（美国伊利诺伊州芝加哥）(SIGMOD' 17) 论文集。计算机协会，美国纽约州纽约市，51–63。https://doi.org/10.1145/3035918.3056100 [32] 弗雷德·B·施奈德。1990. 使用状态机方法实现容错服务：教程。ACM 计算。幸存者。22, 4 (1990 年 12 月), 299–319。https://doi.org/10.1145/98163.98167 [33] Malte Schwarzkopf、Andy Konwinski、Michael Abd-El-Malek 和 John Wilkes. 2013. Omega：适用于大型计算集群的灵活、可扩展的调度程序。第八届 ACM 欧洲计算机系统会议（捷克共和国布拉格）会议记录 (EuroSys '13)。计算机协会，美国纽约州纽约市，351–364。

<https://doi.org/10.1145/2465351.2465386> [34] trillion [n.d.]. Trillian: 总体透明度。 <https://github.com/google/trillian/>。 [35] 罗伯特·范·雷内斯和德尼兹·阿尔廷布肯。 2015. Paxos 变得适度复杂。 ACM 计算。幸存者。 47, 3, 第 42 条 (2015 年 2 月), 36 页。 <https://doi.org/10.1145/2673577> [36] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune 和 John Wilkes。 2015. Google 与 Borg 的大规模集群管理。 第十届欧洲计算机系统会议 (法国波尔多) 会议记录 (EuroSys '15)。 计算机协会, 美国纽约州纽约市, 第 18 条,

17 页。 <https://doi.org/10.1145/2741948.2741964> [37] 张阳, Eman Ramadan, Hesham Mekky 和张志利。 2017. 当 Raft 遇到 SDN: 如何在网络中选举领导者并达成共识。 首届亚太网络研讨会 (中国香港) 论文集 (APNet' 17)。 计算机协会, 美国纽约州纽约市, 1-7。 <https://doi.org/10.1145/3106989.3106999> [38] 郑建军, 林干, 徐家涛, 程伟, 曾楚伟, 杨平安, 张云帆。 2017. PaxosStore: 在微信中实现高可用存储。 过程。 VLDB 捐赠。 10, 12 (2017 年 8 月), 1730-1741。 <https://doi.org/10.14778/3137765.3137778>

Paxos 算法

这总结了我们的简化的 Raft 式 Paxos 算法。红色文本是 Paxos 独有的。

状态

所有服务器上的持久状态：（在响应 RPC 之前在稳定存储上更新）currentTerm 最新术语服务器已看到（首次启动时初始化为 0，单调增加）log[] 日志条目；每个条目包含状态机的命令，以及领导者收到条目时的术语（第一个索引为 1）所有服务器上的易失性状态：commitIndex 已知要提交的最高日志条目的索引（初始化为 0，单调增加）LastApplied 应用于状态机的最高日志条目的索引（初始化为 0，单调增加）候选人的易失性状态：（选举后重新初始化）条目[] 接收到的带有投票的日志条目易失性状态Leaders：（选举后重新初始化）每个服务器的 nextIndex[]，发送到该服务器的下一个日志条目的索引（初始化为领导者提交索引 + 1）每个服务器的 matchIndex[]，已知在服务器上复制的最高日志条目的索引（初始化为 0，单调增加）

追加条目 RPC

由leader调用来复制日志条目；参数：term 领导者的 term prevLogIndex 紧邻新日志条目之前的日志条目索引 prevLogTerm prevLogIndex 条目条目的术语[] 要存储的日志条目（心跳为空；可以发送多个以提高效率）leaderCommit 领导者的 commitIndex 结果：term currentTerm，用于领导者更新自身成功 true 如果跟随者包含与 prevLogIndex - 匹配的条目

dex 和 prevLogTerm
接收器实现：

1. 如果 term < currentTerm，则回复 false 2. 如果日志在 prevLogIndex 处不包含其 term 与 prevLogTerm 匹配的条目，则回复 false 3. 如果现有条目与新条目冲突（索引相同但术语不同），则删除现有条目及其后面的所有条目 4. 附加日志中尚未存在的任何新条目 5. 如果 LeaderCommit > commitIndex：设置 commitIndex = min(leaderCommit, 最后一个新条目的索引条目)

请求投票 RPC

由候选人调用来收集选票 参数：term 候选人的 term leaderCommit 候选人的提交索引 结果：term currentTerm，供候选人更新自己 voteGranted true 表示候选人在 leaderCommit 后收到投票条目[] follower 的日志条目 接收器实现：1. 如果 term < currentTerm 则回复 false 2. 授予投票并在 leaderCommit 后发送任何日志条目

服务器规则

所有服务器：

- 如果 commitIndex > lastApplied：增加 lastApplied 并将 log[lastApplied] 应用到状态机
- 如果 RPC 请求或响应包含 term T > currentTerm：设置 currentTerm = T 并转换为 follower

关注者：

- 回应候选人和领导者的 RPC
- 如果选举超时后没有收到来自当前领导者的 AppendEntries RPC 或向候选人授予投票：转换为候选人

候选人：

- 转换为候选者时，开始选举：将 currentTerm 增加到一个 t，使得 $t \bmod n = s$ ，将 commitIndex 之后的任何日志条目复制到条目[]，并将 RequestVote RPC 发送到所有其他服务器
- 将从 RequestVote 响应收到的任何日志条目添加到条目[]
- 如果从大多数服务器收到投票：通过添加带有 currentTerm 的条目[]来更新日志（如果有多个具有相同索引的条目，则使用具有最大术语的值）并成为领导者

领导：

- 当选后：向每个服务器发送初始的空 AppendEntries RPC（心跳）；在空闲期间重复以防止选举超时
- 如果从客户端收到命令：将条目附加到本地日志，在条目应用于状态机后响应
- 如果关注者的最后一个日志索引为 $\geq \text{nextIndex}$ ：发送 AppendEntries RPC，日志条目从 nextIndex 开始 - 如果成功：更新关注者的 nextIndex 和 matchIndex - 如果 AppendEntries 由于日志不一致而失败：递减 nextIndex 并重试
- 如果存在一个 N，使得 $N > \text{commitIndex}$ 和大多数 matchIndex[i] $\geq N$ ：设置 commitIndex = N

B 筏算法

这是 Raft 论文 [28] 中图 2 的复制品。红色文本是 Raft 特有的。

状态

所有服务器上的持久状态：（在响应 RPC 之前在稳定存储上更新）currentTerm 最新术语服务器已看到（首次启动时初始化为 0，单调增加）votedFor 在当前术语中收到投票的候选者 ID（如果没有则为 null）log[] 日志条目；每个条目包含状态机的命令，以及领导者收到条目时的术语（第一个索引为 1）所有服务器上的易失性状态：commitIndex 已知要提交的最高日志条目的索引（初始化为 0，单调增加）应用于状态机的最高日志条目的最后应用索引（初始化为 0，单调增加）领导者的易失性状态：（选举后重新初始化）每个服务器的 nextIndex[]，下一个日志的索引要发送到该服务器的条目（初始化为领导者最后一个日志索引 + 1）每个服务器的 matchIndex[]，已知在服务器上复制的最高日志条目的索引（初始化为 0，单调增加）

追加条目 RPC

由 leader 调用来复制日志条目；参数：term 领导者的 term prevLogIndex 紧邻新日志条目之前的日志条目索引 prevLogTerm prevLogIndex 条目条目的术语 [] 要存储的日志条目（心跳为空；可以发送多个以提高效率）leaderCommit 领导者的 commitIndex 结果：term currentTerm，用于领导者更新自身成功 true 如果跟随者包含与 prevLogIndex - 匹配的条目

dex 和 prevLogTerm
接收器实现：

1. 如果 term < currentTerm，则回复 false
2. 如果日志在 prevLogIndex 处不包含其 term 与 prevLogTerm 匹配的条目，则回复 false
3. 如果现有条目与新条目冲突（索引相同但术语不同），则删除现有条目及其后面的所有条目
4. 附加日志中尚未存在的任何新条目
5. 如果 LeaderCommit > commitIndex：设置 commitIndex = min(leaderCommit, 最后一个新条目的索引条目)

请求投票 RPC

由候选者调用以收集选票 参数：term 候选者的 term CandidateId 请求投票的候选者 lastLogIndex 候选者最后一个日志条目的索引 lastLogTerm 候选者最后一个日志条目的 term 结果：term currentTerm，候选者更新自己 voteGranted true 表示候选者收到投票 接收者实现：1. 如果 term < currentTerm 则回复 false 2. 如果 votedFor 为 null 或 CandidateId，并且候选者的日志至少与接收者的日志一样最新日志：授予投票

服务器规则

所有服务器：

- 如果 commitIndex > lastApplied：递增 lastApplied，将 log[lastApplied] 应用到状态机
- 如果 RPC 请求或响应包含 term T > currentTerm：设置 currentTerm = T 并转换为 follower

关注者：

- 回应候选人和领导者的 RPC
- 如果选举超时后没有收到来自当前领导者的 AppendEntries RPC 或向候选人授予投票：转换为候选人

候选人：

- 转换为候选人后，开始选举：增加 currentTerm，为自己投票，重置选举计时器并向所有其他服务器发送 RequestVote RPC
- 如果收到大多数服务器的投票：成为领导者
- 如果 AppendEntries RPC 从新的领导者那里收到：转换为追随者
- 如果选举超时：开始新的选举领导人：
- 当选后：向每个服务器发送初始的空 AppendEntries RPC（心跳）；在空闲期间重复以防止选举超时
- 如果从客户端收到命令：将条目附加到本地日志，在条目应用于状态机后响应
- 如果关注者的最后一个日志索引为 $\geq$ nextIndex：发送 AppendEntries RPC，日志条目从 nextIndex 开始 – 如果成功：更新关注者的 nextIndex 和 matchIndex – 如果 AppendEntries 由于日志不一致而失败：递减 nextIndex 并重试
- 如果存在一个 N，使得 $N > \text{commitIndex}$ 和大多数 matchIndex[i] $\geq$ N，以及 log[N].term == currentTerm：设置 commitIndex = N